

Customer Shopping Behavior Analysis

Project Overview

A leading retail company aims to better understand its customers' shopping behavior to improve sales, customer satisfaction, and long-term loyalty. This project analyzes a dataset of **3,900 purchases** to uncover patterns across demographics, product categories, and sales channels.

The goal is to answer: *"How can the company leverage consumer shopping data to identify trends, improve customer engagement, and optimize marketing strategies?"*

Deliverables

1. **Data Preparation (Python):** Cleaning and transforming raw data.
 2. **Database Management (PostgreSQL):** Storing structured data and running complex queries.
 3. **Data Analysis (SQL):** Extracting insights on customer segments, revenue, and trends.
 4. **Dashboard (Power BI):** Visualizing key metrics for stakeholders.
-

Technologies Used

- **Python:** Pandas, NumPy, SQLAlchemy (for Data Cleaning & Database Connection)
 - **SQL (PostgreSQL):** Data Analysis & Querying
 - **Power BI:** Data Visualization & Dashboarding
 - **CSV:** Raw Data Source
-

Step 1: Exploratory Data Analysis & Data Preparation (Python) & Python Connection

Before diving into SQL analysis, we used Python (Pandas) to clean, validate, and structure the raw data. This ensured the insights generated later were accurate and consistent.

-  **Data Loading:**
 - Utilized the **pandas** library to ingest the raw **customer_shopping_behavior.csv** dataset into a DataFrame.
 - Verified the successful load of all **3,900 records and 18 columns**.

- 🔍 **Initial Exploration & Audit:**

- Ran `df.info()` to inspect data types (integers, objects, floats) and identify memory usage.
- Executed `df.describe()` to analyze statistical distributions (mean, min, max) for numerical columns like `Age` and `Purchase Amount`, ensuring values fell within expected ranges.

- ✂ **Missing Data Strategy:**

- Identified missing values in the `Review Rating` column.
- **Action:** Instead of dropping these rows (which would lose valuable sales data), we performed **imputation**. We **calculated the median review rating for each specific product category** and filled the missing values accordingly. This preserved the integrity of the dataset.

- **abc Column Standardization:**

- The raw CSV headers contained spaces and mixed casing (e.g., `Item Purchased`).
- **Action:** Renamed all columns to `snake_case` (e.g., `item_purchased`, `purchase_amount`) to make them compatible with PostgreSQL querying standards and easier to type in code.

- ⚙ **Feature Engineering:**

- **Age Grouping:** Created a new `age_group` column by binning continuous age values into categories: *Young Adult (<25)*, *Adult (25-40)*, *Middle-Aged (41-60)*, and *Senior (>60)*. This allows for demographic segmentation in the final dashboard.
- **Frequency Analysis:** Processed purchase data to derive a `purchase_frequency_days` metric, helping to identify how often customers return.

- ✍ **Data Consistency Check:**

- Analyzed the relationship between `Discount Applied` and `Promo Code Used`.
- **Finding:** These two columns provided redundant information (if a promo code was used, a discount was always applied).
- **Action:** Dropped the `promo_code_used` column to streamline the dataset and reduce redundancy.

- 🛠️ **Database Integration:**
 - Established a connection to the local PostgreSQL database using [SQLAlchemy](#) and [psycopg2](#).
 - Loaded the fully cleaned and transformed DataFrame directly into a SQL table named [customer](#), making it ready for complex querying.

```
df.info() ## check all them coloum info
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   object
3   Item Purchased                       3900 non-null   object
4   Category                            3900 non-null   object
5   Purchase Amount (USD)                3900 non-null   int64
6   Location                             3900 non-null   object
7   Size                                 3900 non-null   object
8   Color                               3900 non-null   object
9   Season                              3900 non-null   object
10  Review Rating                        3863 non-null   float64
11  Subscription Status                  3900 non-null   object
12  Shipping Type                        3900 non-null   object
13  Discount Applied                     3900 non-null   object
14  Promo Code Used                      3900 non-null   object
15  Previous Purchases                    3900 non-null   int64
16  Payment Method                       3900 non-null   object
17  Frequency of Purchases                3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

```
# df.describe() ## describe show the entire aggerigation function on the df dataset in numerical coloums
df.describe(include = 'all') ## entire aggregation function with numerical and catgorical coloums
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900	3900.000000	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2	NaN	6	7
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No	NaN	PayPal	Every 3 Months
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223	NaN	677	584
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN	25.351538	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN	14.447125	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN	1.000000	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN	13.000000	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN	25.000000	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN	38.000000	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN	50.000000	NaN	NaN

```
df.isnull().sum() ## check null value and sum them
```

Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	37
Subscription Status	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0
Payment Method	0
Frequency of Purchases	0
dtype: int64	

Connecting Python to PostgreSQL

To analyze the data in SQL, we established a connection between our Pandas DataFrame and the Postgres database using the `sqlalchemy` library.

Definition: The `create_engine` function from SQLAlchemy creates a connection object that allows Python to communicate with the database. We then use `df.to_sql()` to push our cleaned dataset directly into a SQL table.

Packeages need to be download

```
pip install sqlalchemy psycopg2
# or
uv add sqlalchemy psycopg2
```

Code Snippet:

```
from sqlalchemy import create_engine
username = "postgres"          ## default user
password = "1234"             ## installation password
host = "localhost"            ## if running locally
port = '5432'                  ## local default port number
database = "customer_behavior" ## the database name that you are going to
                                connect

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:
{port}/{database}")           # Create the connection engine

## step 2 load dataframe into database/ postgresQL

table_name = "customer"
df.to_sql(table_name, engine, if_exists = "replace", index = False)

display(f"Data successfully loaded into table {table_name} in database
{database}.")
```

 Screenshot: Python to PostgreSQL Connection Execution

Database interaction

```
## step 1: connect to PostgreSQL
## replace placeholder with you actual credential

# from sqlalchemy import create_engine

username = "postgres"          ## default user
password = "1234"              ## installation password
host = "localhost"             ## if running locally
port = '5432'                  ## local default port number
database = "customer_behavior" ## the database name that you are going to connect

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}") # Create the connection engine

## step 2 load dataframe into database/ postgresQL

table_name = "customer"
df.to_sql(table_name, engine, if_exists = "replace", index = False)

display(f"Data successfully loaded into table {table_name} in database {database}.")
```

Python

'Data successfully loaded into table customer in database customer_behavior.'

Step 2: Data Analysis (SQL)

Below are the SQL queries used to solve specific business questions.

Q1: What is the total revenue generated by male vs female customers? Analyzing spending power by gender.

```
SELECT gender, SUM(purchase_amount) AS revenue
FROM customer
GROUP BY gender;
```

🖱️ Output:

	gender 	revenue 
	text	numeric
1	Female	75191
2	Male	157890

Q2: Which customers used a discount but still spent more than the average purchase amount? Identifying high-value customers who are responsive to discounts.

```
SELECT customer_id, discount_applied, purchase_amount
FROM customer
WHERE discount_applied = 'Yes'
AND purchase_amount > (SELECT AVG(purchase_amount) FROM customer);
```

👉 Output:

	customer_id bigint	discount_applied text	purchase_amount bigint
1	2	Yes	64
2	3	Yes	73
3	4	Yes	90
4	7	Yes	85
5	9	Yes	97
6	12	Yes	68
7	13	Yes	72
8	16	Yes	81
9	20	Yes	90
10	22	Yes	62
11	24	Yes	88
12	29	Yes	94
13	32	Yes	79
14	33	Yes	67
15	35	Yes	91
16	37	Yes	69
17	40	Yes	60
18	41	Yes	76
19	43	Yes	100
20	44	Yes	69
21	55	Yes	94
22	57	Yes	73

Q3: What are the top 5 products with the highest average review rating? Understanding which products satisfy customers the most.

```
SELECT item_purchased, ROUND(AVG(review_rating::numeric),2) AS "Average Product Rating"
FROM customer
```

```
GROUP BY item_purchased
ORDER BY AVG(review_rating) DESC
LIMIT 5;
```

🖱️ Output:

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

Q4: Compare the average purchase amounts between Standard and Express shipping.
Does shipping speed correlate with higher spending?

```
SELECT shipping_type, ROUND(AVG(purchase_amount),2) AS "Average Purchase Amount"
FROM customer
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type;
```

🖱️ Output:

	shipping_type text	Avgerage Purchase amount numeric
1	Standard	58.46
2	Express	60.48

Q5: Do subscribed customers spend more? Comparing average spend and total revenue between subscribers and non-subscribers.

```
SELECT COUNT(customer_id) AS "Total Customer",
       subscription_status,
       ROUND(AVG(purchase_amount),2) AS avg_spend,
       SUM(purchase_amount) AS revenue
FROM customer
```



```
GROUP BY subscription_status
ORDER BY avg_spend, revenue DESC;
```

🖱 Output:

	Total Customer bigint	subscription_status text	avg_spend numeric	revenue numeric
1	1053	Yes	59.49	62645
2	2847	No	59.87	170436

Q6: Which 5 products have the highest percentage of purchases with a discount applied?
Identifying products that rely heavily on promotions.

```
SELECT item_purchased,
       ROUND(100 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END) /
COUNT(*), 2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

🖱 Output:

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.00
3	Coat	49.00
4	Sweater	48.00
5	Pants	47.00

Q7: Segment customers into New, Returning, and Loyal. Segmentation based on the number of previous purchases.

```
WITH customer_type AS (
  SELECT customer_id, previous_purchases,
```

```

CASE
    WHEN previous_purchases = 1 THEN 'New'
    WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
    ELSE 'Loyal'
END AS customer_segment
FROM customer
)
SELECT customer_segment, COUNT(*) AS "Number of Customer"
FROM customer_type
GROUP BY customer_segment;

```

👉 Output:

	customer_segment text	Number of Customer bigint
1	Loyal	3116
2	New	83
3	Returning	701

Q8: What are the top 3 most purchased products within each category? Using Window Functions to rank items.

```

WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_order,
           ROW_NUMBER() OVER(PARTITION BY category ORDER BY COUNT(customer_id)
                               DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank, category, item_purchased, total_order
FROM item_counts
WHERE item_rank <= 3;

```

👉 Output:

	item_rank bigint	category text	item_purchased text	total_order bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

Q9: Are repeat buyers (more than 5 previous purchases) more likely to subscribe?

Checking the correlation between loyalty and subscription status.

```
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

👉 Output:

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

Q10: Revenue by Age Group. Calculated total revenue contribution of each demographic bucket.

```
SELECT
  CASE
    WHEN age < 25 THEN 'Young Adult'
    WHEN age BETWEEN 25 AND 40 THEN 'Adult'
    WHEN age BETWEEN 41 AND 60 THEN 'Middle-aged'
    ELSE 'Senior'
  END AS age_group,
  SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY 1
ORDER BY total_revenue DESC;
```

👉 Output:

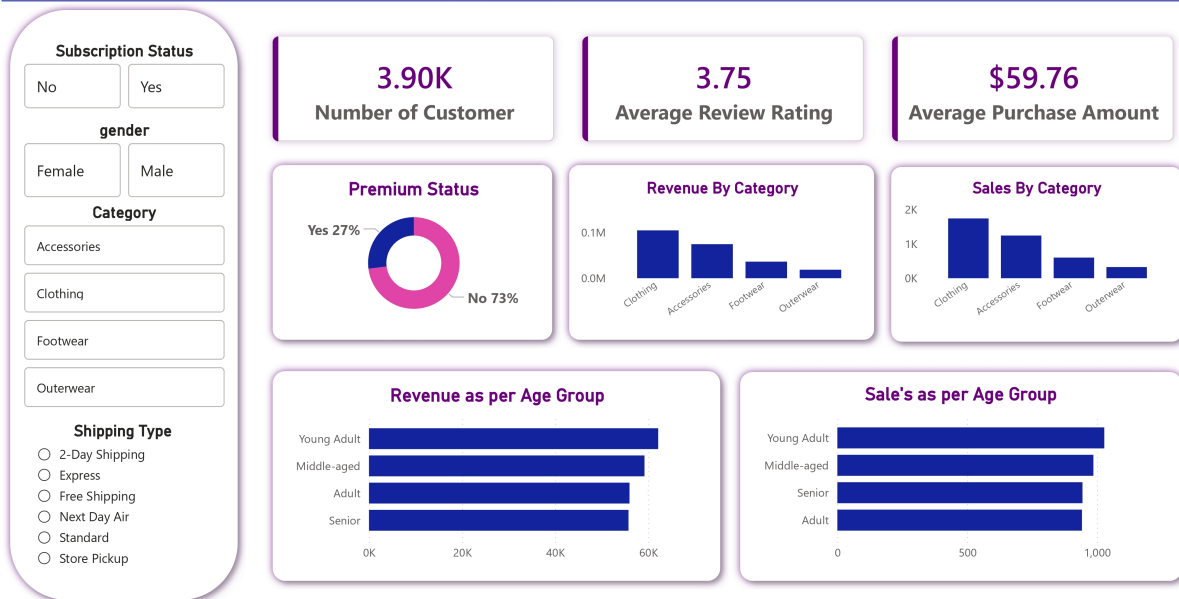
	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

Step 3: Power BI Dashboard Guide

The dashboard provides an interactive view of the analysis. Below is the step-by-step process used to build it.

👉 **Dashboard Preview:**

Customer Shopping Behavior Dashboard



Step-by-Step Creation Guide: * Import Data:

* Opened Power BI Desktop.

* Selected Get Data > Text/CSV and loaded
`customer\_shopping\_behavior.csv`.

• Data Transformation (Power Query):

- Checked for null values in the Review Rating column.
- Ensured data types were correct (e.g., Purchase Amount as Currency, Age as Whole Number).
- Created a new conditional column for **Age Groups** (Young Adult, Adult, Middle-Aged, Senior) to match the SQL analysis.

• Building Key Measures (DAX):

- Created measures for Total Customers, Average Purchase Amount, and Average Rating.

• Creating Visuals:

- **KPI Cards:** Added top-row cards for quick insights (Total Customers: 3.9K, Avg Purchase: \$59.76).

- **Donut Chart:** Used to visualize **Subscription Status** (Yes vs. No) showing 73% non-subscribers.
- **Bar Charts:**
 - **Revenue by Category:** Shows Clothing is the highest earning category.
 - **Sales by Age Group:** Highlights that "Adults" and "Young Adults" drive sales.
- **Slicers/Filters:** Added slicers for **Gender**, **Category**, and **Shipping Type** to allow users to filter the data dynamically.
- **Formatting & Design:**
 - Applied a consistent color theme (Blue/Gold/Grey).
 - Aligned charts using the Grid layout.
 - Added the project title and borders for a clean UI.

Key Business Insights

1. **Subscription Opportunity:** Only **27%** of customers are subscribers. Repeat buyers are a prime target for subscription upsells.
2. **Demographics:** The "Adult" and "Middle-aged" groups contribute the most revenue.
3. **Product Trends:** "Clothing" is the dominant category, while specific items like "Jewelry" and "Pants" are top sellers within their categories.
4. **Discount Efficiency:** High-value customers still use discounts; targeted promotions could increase basket size further.

Contact

Author: **Ritesh Brahmachari** GitHub: [GitHub](#) linkedIn: [LinkedIn](#)